Project #11 Instructions
11/5/26
Chris Grimm

Assignment Instructions

Week 9 Project: Advanced Canvas

Start this project as early as possible. This week's material requires more independent work, so plan time to read, experiment, and ask questions as needed.

You will complete one project that combines skills you have already developed with the new canvas concepts introduced this week.

Project Requirements

Create a stand-alone project that demonstrates **at least five of the following six topics**:

- **Display part of an image on a canvas.** See *Slicing, Dicing, and Resizing an Image*, p. 278.
- **Draw at least five separate segments of text on a canvas.** See *Drawing Text*, pp. 279–280.
- **Fill a shape with a pattern.** See *Filling Shapes with Patterns*, pp. 283–293.
- **Make a shape interactive.** See *Making Your Shapes Interactive*, pp. 293–300.
- **Add animation.** See *Animating the Canvas*, pp. 300–307.
- **Modify the Maze Game in at least five ways.** See *A Practical Example: The Maze Game*, pp. 307–315.

Organize Your Project

You may demonstrate the selected topics on one canvas or across multiple canvases and pages. Placing each topic on its own page may make your code easier to manage.

If you create additional pages, include:

- A link from your homepage to each project page.
- A link from each project page back to your homepage.

Before submitting, test your links and make sure all project features work on your published website.

Submit Your Work

Submit the following through the **Week 9 Project** link in Blackboard:

1. **Your homepage URL**, with links to all required project work.
2. **At least three sentences for each selected topic**, explaining what you created, how it demonstrates the topic, and what you learned or changed while developing it.

Please reach out with any questions. I am happy to help!

Sources:

Img/

brick_title.gif, dawin.jpg

Animation.html, CroppedPicture.html, home.html, InteractiveCircles.html, Pattern.html, script.js, style.css, and Text.html

Live URL: http://157.245.246.41:8014/Project11/home.html

1: Display part of an image on a canvas. I created a canvas that pulls a source image of Charles Darwin from the Img folder and selectively renders only a specific cropped portion of his face. This demonstrates the topic of image slicing by using the overloaded drawImage() method, passing nine distinct arguments to define the source coordinates and the destination placement. While developing this, I learned that keeping the original image tag hidden via CSS ensures the canvas logic can still reference the loaded DOM element without awkwardly displaying the uncropped source image on the page.

2: Draw at least five separate segments of text on a canvas. I generated a single canvas that sequentially renders a numbered list of five distinct text segments, each utilizing a different styling technique. This demonstrates the topic by utilizing fillText(), strokeText(), and

manipulating the context.font property to adjust families and sizes dynamically. During development, I changed the fourth and fifth segments to experiment with drop shadows, learning that I must explicitly reset the shadow properties back to transparent afterward so they do not accidentally inherit onto the final text string.

3: Fill a shape with a pattern. I created a canvas that draws a rectangle and completely fills its interior with a repeating brick tile image from the Img folder. This demonstrates pattern filling by utilizing the context.createPattern() method, passing in the hidden DOM image and the "repeat" string argument. I learned that once a pattern is created, it is treated exactly like a solid hex color by the canvas engine, meaning I could simply assign it to context.fillStyle before invoking the fill method on my path.

4: Make a shape interactive. I built an application where clicking a button spawns randomly sized circles, and clicking directly on the canvas highlights a specific circle with a thick white border. This demonstrates shape interactivity by calculating the exact mathematical distance between the user's raw mouse click coordinates and the center point of each generated circle using the Pythagorean theorem. I changed the textbook's default colors to match my dark-slate theme and learned that storing shape data in a JavaScript array is strictly required for interactivity, as the canvas itself immediately forgets shapes once they are drawn.

5: Add animation. I developed a physics-based animation loop that spawns colored balls that drop via simulated gravity, bounce off the canvas walls, and gradually slow down due to a friction multiplier. This demonstrates the topic of canvas animation by using setTimeout() to recursively trigger a frame-drawing function, clearing the canvas completely before redrawing the balls in their new coordinate positions. I changed the visual styling to match my project theme and learned that true canvas animation does not involve moving objects, but rather rapidly erasing and redrawing the entire frame to simulate motion based on state variables.